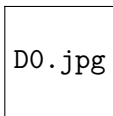


Lecture 30: Functional Dependencies

Z2004: Database Management Systems

Dr. Innocent Nyalala



IIT Madras Zanzibar

18 May 2026

Outline

- ▶ The messy table problem
- ▶ Three anomalies
- ▶ What is a Functional Dependency?
- ▶ FD examples from our table
- ▶ Trivial vs. non-trivial FDs
- ▶ Attribute closure
- ▶ Finding candidate keys
- ▶ Think-Pair-Share
- ▶ Summary

Module 7 — Normalisation

This lecture opens the normalisation unit. FDs are the **foundation** — everything in L31–L33 builds on today.

The Messy Table — A Disaster Waiting to Happen

SID	SName	SEmail	CID	CName	IID	IName	Grade	Sem
S001	Amani	amani@st	CS101	Databases	I01	Dr. Ouma	A	Sp26
S002	Baraka	baraka@st	CS101	Databases	I01	Dr. Ouma	B	Sp26
S003	Chidi	chidi@st	CS202	Algorithms	I02	Dr. Said	A	Sp26
S001	Amani	amani@st	CS202	Algorithms	I02	Dr. Said	C	Sp26
S004	Dina	dina@st	CS101	Database	I01	Dr. Ouma	B+	Sp26
S005	Emil	emil@st	CS202	Algorithms	I03	Dr. Said	A-	Sp26

"Databases" vs "Database"!
Duplicated 3×!

This table is a disaster waiting to happen

9 attributes, endless redundancy. InstructorName stored in every row. One typo creates inconsistency.

Three Anomalies of Bad Design

Insertion Anomaly

Want to add new course
CS303: OS?

⚠ *No student enrolled
yet — can't insert!*

CourseID and CourseName
trapped inside a student row.

Deletion Anomaly

Chidi (S003) drops
CS202 (only student)?

⚠ *We lose CS202 and
Dr. Said's info too!*

Course data tied to
student enrolment rows.

Update Anomaly

Dr. Ouma changes name
to Dr. Abubakar?

⚠ *Must update 3 rows.
Miss one = corrupt!*

Every course section
stores instructor name.

Root Cause

Data that **belongs together logically** is scattered across multiple rows. The fix: understand **Functional Dependencies** and use them to split tables properly.

What Is a Functional Dependency?

Definition

$X \rightarrow Y$ means: **knowing X uniquely determines Y**.

For every pair of tuples t_1, t_2 :

if $t_1[X] = t_2[X]$ then $t_1[Y] = t_2[Y]$

Intuition: X is the “key”, Y is the “value”

► **StudentID $\rightarrow$ StudentName**

Same ID $\Rightarrow$ same name. Always.

► **CourseID $\rightarrow$ CourseName**

Same course code $\Rightarrow$ same name.

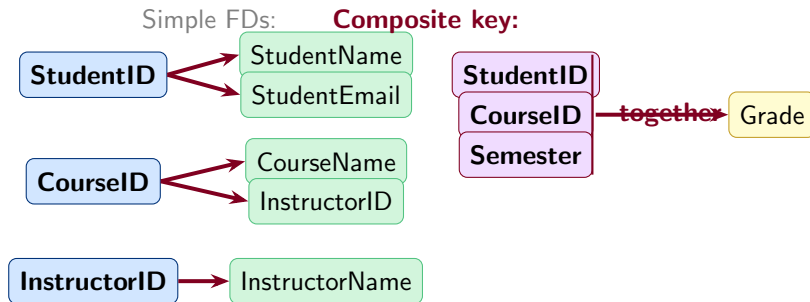
► **Email $\nrightarrow$ StudentID?**

Two students could share an email (family!)



✓ Same ID $\Rightarrow$ Same name

Functional Dependencies in StudentEnrolment



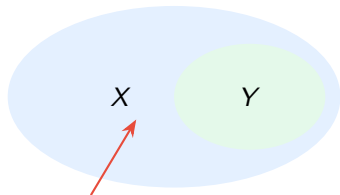
Key Insight

Grade needs all three — knowing only **StudentID** or only **CourseID** is not enough.

Why This Matters

These FDs tell us **how to split the table**. Each FD group → its own relation.

Trivial vs. Non-Trivial Functional Dependencies



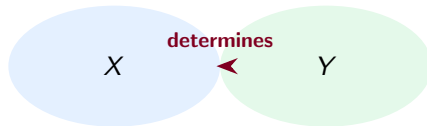
TRIVIAL FD

$X \rightarrow Y$ where $Y \subseteq X$

Always true, says nothing.

e.g. $\{SID, CID\} \rightarrow SID$

Of course! SID is in the set.



NON-TRIVIAL FD

$X \rightarrow Y$ where $Y \not\subseteq X$

Says something meaningful!

e.g. $CourseID \rightarrow CourseName$

One key, different attribute.

Rule of Thumb

In normalisation we only care about **non-trivial FDs**. Trivial ones are always satisfied and tell us nothing about the schema.

Attribute Closure: What Can We Infer?

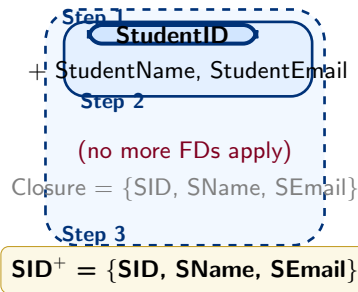
Question: Starting with {StudentID}, what can we determine?

Given FDs:

- ▶ $\text{StudentID} \rightarrow \text{StudentName}, \text{StudentEmail}$
- ▶ $\text{CourseID} \rightarrow \text{CourseName}, \text{InstructorID}$
- ▶ $\text{InstructorID} \rightarrow \text{InstructorName}$
- ▶ $\{\text{SID}, \text{CID}, \text{Sem}\} \rightarrow \text{Grade}$

Algorithm (closure^+ of X):

- 1 Start: $\text{result} = X$
- 2 Scan all FDs: if left side $\subseteq \text{result}$, add right side
- 3 Repeat until no change



Notation

$\{\text{StudentID}\}^+ = \text{all attributes determined by StudentID (under the given FD set)}$

Finding Candidate Keys Using Closure

Candidate Key Test

X is a **superkey** if $X^+ = \text{all attributes}$.

X is a **candidate key** if it is a **minimal** superkey (remove any attribute and it no longer determines everything).

Test: {StudentID, CourseID, Semester}

- 1 Compute closure...
- 2 $\text{SID} \rightarrow \text{SName, SEmail}$
- 3 $\text{CID} \rightarrow \text{CName, IID}$
- 4 $\text{IID} \rightarrow \text{IName}$
- 5 $\{\text{SID, CID, Sem}\} \rightarrow \text{Grade}$
- 6 **Result:** all 9 attributes ✓

{SID, CID, Semester}



SID, SName, SEmail

CID, CName, IID

IName, Grade

✓ All 9 attrs!

⇒ **Candidate Key**

Think-Pair-Share (4 minutes)

Given these FDs — find all candidate keys

Relation $R(A, B, C, D, E)$

- ▶ $A \rightarrow BC$
- ▶ $CD \rightarrow E$
- ▶ $B \rightarrow D$
- ▶ $E \rightarrow A$

💡 Compute closure of $\{A\}$:

💡 Does $\{A\}$ determine everything?

💡 Try other sets.

💡 Which sets are **minimal**?

Hint

There may be **more than one** candidate key!

Summary

Key Concepts Today

- ▶ **Bad design** $\Rightarrow$ 3 anomalies
- ▶ $X \rightarrow Y$: knowing X gives Y
- ▶ **Trivial** FD: $Y \subseteq X$ (boring)
- ▶ **Non-trivial**: meaningful dependency
- ▶ **Closure** X^+ : all attrs from X
- ▶ **Candidate key**: minimal superkey

Find FDs

(observe data)



Compute Closures

Armstrong's axioms (L31)



Find Candidate Keys

then normalise (L32–L33)

Next Lecture — L31: Armstrong's Axioms

Formal inference rules to derive *all* valid FDs from a given set. Foundation of the canonical cover algorithm.